

The μ OS++ test framework

Install, build, run and add tests, step by step

Kernel: micro-os-plus-iii

Ports: aarch32, aarch64, cortexm, posix-arch

The μ OS++ test framework, step by step

- 1. What the framework is
- 2. Install, once
- 3. Build one configuration
- 4. Run tests
- 5. Add a test to a board
- 6. Add a harness suite
- 7. Run a harness suite as a cortexm board app
- 8. Add a platform or a configuration
- 9. When something goes wrong
- 10. What exists today
- 11. Rules to keep

The μOS++ test framework, step by step

This guide explains how to install the test framework, build it, run the tests, and add new ones. Each step says **what** you do and **how** you do it. Every command here was checked against the code.

1. What the framework is

The framework has four parts:

- **CTest** registers every test and runs it.
- **CMake** builds the test images.
- **xpm**, the xPack tool, gives each build its own pinned compiler and tools, and has named commands for every step. These are called **actions**.
- **VS Code**, with the xPack plugin, shows those actions as buttons.

All of it lives in one folder: `micro-os-plus-iii-smp.git/tests/`.

A test tells the framework its result in one of two ways:

- it prints a line `RESULT: PASS`, `RESULT: FAIL` or `RESULT: SKIP`;
- or it exits with a code, where 0 means pass.

Three files, all edited by hand, decide what exists:

File	What it decides
<code>tests/package.json</code>	The configurations : which platform, compiler and build type. Also the actions , the commands you run.
<code>tests/cmake/tests-main.cmake</code>	Which port a platform tests, chosen by the platform name's prefix.
<code>tests/platforms/<platform>/CMakeLists.txt</code>	Which tests that platform registers in CTest.

Words used below

Word	Meaning
platform	One board or emulator setup, e.g. <code>aarch32-rpi-zero-2w</code> , <code>cortexm-pico2</code> , <code>native</code> .
configuration (<code>C</code>)	A platform plus a compiler plus a build type, e.g. <code>cortexm-pico2-cmake-gcc-debug</code> . Each one has its own folder, <code>build/C/</code> .
case	One CTest test, named <code><platform>-<test>-<variant></code> .
variant	Where the test runs. <code>qemu</code> runs in the emulator, <code>host</code> runs on the PC, and <code>hwd</code> runs on the real board.
action	A named command in <code>package.json</code> , run as <code>xpm run <action> --config C</code> .
board test	A test that belongs to one board. It lives in the port: <code><port>/test/<board>/<test>/</code> .
harness suite	A portable test with no <code>main()</code> . It lives in <code>tests/sources/<suite>/</code> and runs on many platforms.

2. Install, once

2.1 The workspace folder

What: one folder that holds the six repositories side by side. The default is `~/Work`, but any folder works (this machine also uses `~/TMP`).

How:

```
mkdir -p ~/Work && cd ~/Work
for r in aarch32 aarch64 cortexm devices posix-arch smp; do
  git clone "$HOME/Downloads/GIT/micro-os-plus-iii-$r.git" \
    "micro-os-plus-iii-$r.git"
done
```

Keep the `.git` at the end of each folder name. The framework looks for the ports under exactly those names (`../../micro-os-plus-iii-<port>.git`). No symlinks are needed.

2.2 The tools

What: Node packages, plus the xPack packages CMake, Ninja, QEMU, the build helper and the test libraries.

How:

```
cd ~/Work/micro-os-plus-iii-smp.git/tests
npm install
xpm install
```

All the remaining commands in this guide run from this `tests/` folder.

3. Build one configuration

What: four steps, always in this order:

Step	Command	What it does
1	<code>xpm run install --config C</code>	Puts the pinned compiler into <code>build/C/xpacks/</code> .
2	<code>xpm run prepare --config C</code>	Runs CMake with that compiler and finds the tests.
3	<code>xpm run build --config C</code>	Compiles every test image.
4	<code>xpm run test --config C</code>	Runs every emulated and host test. It skips hardware tests.

Example:

```
C=aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run install --config $C
xpm run prepare --config $C
xpm run build --config $C
xpm run test --config $C
```

Why the order matters:

- `prepare` needs the compiler that `install` puts there. If you skip `install`, CMake picks the PC's own compiler (`found /usr/bin/cc`).
- Only `prepare` tells CMake which compiler to use. Skip it and `build` may use the wrong one.
- `test` does **not** build. Run it on a tree that was never built and every test fails with `no *-host executables`.

Debug and release are separate. Each has its own `build/C/` folder, so a `-release` configuration needs its own `install`, too.

Never delete `build/C/`, because it holds the pinned compiler. If CMake got confused, delete only `build/C/CMakeCache.txt` and run `prepare` again.

4. Run tests

I want to...	Command
run all emulated and host tests of one configuration	<code>xpm run test --config C</code> (build first)
run one test; it builds whatever it needs first	<code>xpm run test-<test>-<variant> --config C</code>
see which tests exist	<code>PATH="\$PWD/build/C/xpacks/.bin:\$PATH" ctest --test-dir build/C -N</code>
run one case by hand	same <code>PATH</code> , then <code>ctest --test-dir build/C -V -R <platform>-<test>-<variant></code>
run debug and release in one go	<code>xpm run test-<platform>-cmake</code> . This exists for the upstream platforms, <code>aarch32-rpi-zero-2w</code> , <code>aarch64-rpi-zero-2w</code> and <code>native</code> ; <code>test-cortex-cmake</code> runs the four <code>qemu-cortex-m*</code> ones.

Examples:

```
xpm run test-smp_test0-qemu      --config aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run test-rtos-apis-qemu     --config cortexm-pico2-cmake-gcc-debug
xpm run test-cmsis-os-validator-host --config native-cmake-gcc-debug
```

From VS Code: open `tests/` and pick a configuration in the xPack view. The plugin shows **actions**, not CTest cases. A test appears there only if its `test-*` action exists in `package.json`.

On real hardware (`hwd`):

- The board must be connected.
- Run one test per power cycle.
- The test goes through OpenOCD only, with no GDB. The result comes over the UART or semihosting.
- The generic `test` action skips `hwd` cases on purpose. Run a hardware case by its own action, e.g. `xpm run test-smp_test0-hwd --config aarch32-rpi-zero-2w-cmake-gcc-debug`.

Emulated suites run one at a time, on a quiet machine. When several run in parallel they get slow and can time out for no real reason.

Where the logs are: next to the images, in `.qemu-logs/`, `.host-logs/` or `.hw-logs/`.

5. Add a test to a board

What: a new folder with one test in the port, then an action so VS Code shows it. The platform registers the test by **scanning the port's test folders** when `prepare` runs. You do **not** edit any CMake file in the framework.

How:

1. **Write it.** Create `<port>/test/<board>/<name>/main.cpp`, starting from a sibling test. It must print `RESULT: PASS` or `RESULT: FAIL`. On hardware it must also **stop** after that line: call `hw_result::ok()` / `hw_result::fail()` (cortexm) or `semihosting::report_result()`. Otherwise the run never ends.
2. **Declare what it needs**, only if it differs from the default. Add it to the lists in `<port>/test/<board>/tests.cmake`:
`BOARD_TEST_NEED_DEVICES`, `BOARD_TEST_SELF_CONTAINED`, `BOARD_TEST_NO_KERNEL`, and on cortexm `BOARD_TEST_HWD_ONLY` (no emulator image).
3. **Give it time**, only if it runs longer than 300 s in QEMU. Add it to `timeout_for` in `micro-os-plus-iii-smp.git/test_smpl/run-qemu.sh`.
4. **Commit it.** If you wrote it in another copy of the port, push it there, then `git pull` in the workspace copy.
5. **Re-scan:** `xpm run prepare --config C`. Without this step the test does not exist in CTest.
6. **Add the action.** Run `python3 ~/.claude/skills/micro-os-xpack-tests/check-actions.py . C --emit`, which prints the missing `test-<name>-<variant>` lines. Paste them into the `actions` of **every** configuration that builds the test. Nothing generates these lines for you.
7. **Check and run:** `sh python3 ~/.claude/skills/micro-os-xpack-tests/check-actions.py . PATH="$PWD/build/C/xpacks/.bin:$PATH" cmake --build build/C -- -k 0 xpm run test-<name>-qemu --config C` The full build matters. Every per-test action builds the **whole** tree, so one broken test breaks every action of that configuration.

After adding the test, run the emulated set of the other boards on the same port again. A new test must not change their results.

6. Add a harness suite

What: a portable test that runs on many platforms. It has no `main()`. Its entry point is `int os_main(int argc, char* argv[])`, and it returns 0 for pass.

How:

1. **Create** `tests/sources/<name>/`, copying `mutex-stress`:
2. `CMakeLists.txt` defines the library `test::<name>`;
3. `src/*.cpp` holds the test;
4. `include/cmsis-plus/os-app-config.h` holds its RTOS settings.
5. **Make it known to a platform.** Add `"${CMAKE_SOURCE_DIR}/sources/<name>"` to `tests/platforms/<platform>/cmake/dependencies-folders.cmake`.
6. **Register it.** Add `add_harness_suite(<name> <name>-test)` to that platform's `CMakeLists.txt`. The upstream platforms use `add_test_executable` instead.
7. **Re-scan, add the action and run it**, as in steps 5–7 of section 5.

The suite links `micro-os-plus::platform-support`, which supplies `main()` and the start-up hooks. A board test must **not** link it, because it has its own `main()`.

Suites today: `mutex-stress`, `rtos-apis`, `cmsis-os-validator`, plus `blinky` and `instrumentation` (`nucleo-f411re` only). The validator raises NVIC IRQ 0: on Cortex-M that is the NVIC, on `native` the validator xpack's signal shims, and on the four Raspberry Pi platforms the BCM2837's local Mailbox 1 (see `TESTS-CATALOG.md` §4).

7. Run a harness suite as a cortexm board app

What: run a suite on a cortexm board, on the real board and, where the board has a QEMU image, in QEMU too. `cortexm-pico2`, `cortexm-nucleof411`, `cortexm-weactf411` and `cortexm-weactf412` already do this for `rtos-apis`, `mutex-stress` and `cmsis-os-validator`. To do it on another board, copy these five pieces. **All five are needed.**

1. **The verdict wrapper, once per board**, `test/<board>/harness-suite.hpp`, and **a folder per suite**, `test/<board>/<suite>/harness-suite.cpp`, which only includes `../harness-suite.hpp`. The folder name becomes the app name. The header defines `__wrap_os_main()`: it runs the suite, prints `RESULT: PASS` or `RESULT: FAIL`, and returns the code. (The Pi ports keep the wrapper in each suite's `harness-suite.cpp` instead.)
 2. `board_test_libs()` in `tests.cmake` returns the suite library, its extras, and an interface target (on pico2, `pico2-harness-suite`). That target carries:
 3. the defines `OS_USE_OS_APP_CONFIG_H` `UOS_HARNESS_SUITE` `_POSIX_C_SOURCE=200809L` `_GNU_SOURCE`;
 4. the platform's `include/` folder;
 5. the semihosting, newlib and POSIX-io libraries;
 6. the link option `-Wl,--wrap=os_main`.
 7. **The board's `os-app-config.h`** hands over to the suite's own settings when `UOS_HARNESS_SUITE` is set. Each of the board's own values is wrapped in `#ifndef`.
 8. **The port's builder** already builds the emulated image with one CPU, because the generic QEMU core is single-core. You don't need to change it.
 9. **One action per case** in `package.json`, as in step 6 of section 5.
-

8. Add a platform or a configuration

A platform:

1. Copy the closest folder in `tests/platforms/`.
2. The name must start with a prefix that `tests/cmake/tests-main.cmake` knows: `aarch32`, `aarch64`, `native`, `cortexm`, `pico2` or `2xcortex`. Otherwise add a new branch there.
3. Set the board in `cmake/definitions.cmake`: `set (BOARD "<id>" CACHE STRING "" FORCE)`.

A port platform's `CMakeLists.txt` does five things:

1. loads the port's board facts;
2. reuses the port's own test builder;
3. scans `<port>/test/<board>/*`;
4. registers one case per test and variant;
5. adds the harness suites.

A configuration:

1. In `tests/package.json`, copy the nearest debug and release pair.
2. Change `platformName`, `toolchainFileName` and `shortConfigurationName`. The short name must be unique.
3. Pick the tools it inherits: `qemu-arm-dependencies` for an emulator, `openocd-dependencies` for a board.
4. Set its `test` action to `ctest -V -LE hwd`.
5. Add its `test-*` actions. If it has emulated tests, also add a top-level `test-<platform>-cmake`.

The release entry only inherits the debug one and changes `buildType`.

Then follow section 3 with `install → prepare → build → test`.

9. When something goes wrong

What you see	Why, and the fix
<code>found /usr/bin/cc</code> , or the compiler must be 15.2	CMake ran before <code>install</code> , or <code>build</code> ran before <code>prepare</code> . Run <code>install</code> , then <code>prepare</code> .
Missing <code>.../build/C/xpacks/.../CMakeLists.txt</code> on a release configuration	The release folder never had its own <code>install</code> .
<code>no *-host executables</code> , or every case fails	<code>test</code> does not build. Run <code>build</code> first, or use a per-test action.
The test runs in CTest but is missing in VS Code	Its <code>test-*</code> action is missing. Run <code>check-actions.py</code> .
Every <code>test-*</code> action of one configuration fails while building	One test in that tree does not compile. Run <code>cmake --build build/C -- -k 0</code> to see them all.
A new port test is missing from <code>ctest -N</code>	You didn't run <code>prepare</code> again, the folder has no <code>.cpp</code> file, or the workspace copy wasn't pulled.
<code>alias micro-os-plus::iii already exists</code>	The kernel was added twice. The port already adds it.
<code>undefined reference to os_startup_initialize_hardware...</code>	A suite is missing <code>platform-support</code> .
A QEMU case times out with no error	Tests ran in parallel, or the test needs a <code>timeout_for</code> entry. Run it again alone first.
A hardware test never ends	It prints <code>RESULT</code> but never stops. Add <code>hw_result</code> or <code>report_result</code> .

10. What exists today

There are **22 platforms** and **70 configurations** (35 debug + 35 release; `native` has one configuration per compiler). The numbers below are the debug case counts from `ctest -N`. What every test does, its scheduling mode and the probe each board uses are in `TESTS-CATALOG.md`.

Platform	Tests	Emulated / host	Hardware
<code>aarch32-rpi-zero-2w</code>	12 port tests + <code>mutex-stress</code> , <code>rtos-apis</code> , <code>cmsis-os-validator</code>	15 qemu	15
<code>aarch64-rpi-zero-2w</code>	the same	15 qemu	15
<code>aarch32-rpi3b</code>	the same	15 qemu	14 (no <code>usb_test</code> : the Pi 3 B's USB is behind a hub)
<code>aarch64-rpi3b</code>	the same	15 qemu	14 (likewise)
<code>aarch32-luckfox-lyra</code>	19 port tests + <code>mutex-stress</code>	–	20
<code>native</code> (POSIX, on the PC)	13 port tests + <code>cmsis-os-validator</code>	14 host	–
<code>cortexm-pico2</code>	12 port tests + the 3 suites as board apps	5 qemu	15
<code>cortexm-pico2-rp2350b-psram</code>	14 port tests	2 qemu	14
<code>cortexm-pico2-pizero</code>	14 port tests	–	14
<code>cortexm-nucleof411</code>	<code>mos-test1</code> + the 3 suites	–	4
<code>cortexm-weactf411</code>	<code>mos-test1</code> , <code>spi-pipeline</code> + the 3 suites	–	5
<code>cortexm-weactf412</code>	<code>mos-test1</code> , <code>uart-test1</code> + the 3 suites	–	5
<code>pico2-1cpu</code> (1 × Cortex-M33)	the 3 suites	3 qemu	–
<code>2xcortex-m33</code> (2 × Cortex-M33, SMP)	the 3 suites	3 qemu	–
<code>qemu-cortex-m0 / m3 / m4f / m7f</code>	the 3 suites	3 qemu each	–
<code>nucleo-f411re</code>	3 suites + <code>blinky</code> , <code>instrumentation</code>	–	4
<code>nucleo-f767zi</code> , <code>nucleo-h743zi</code> , <code>raspberrypi-pico</code>	the 3 suites	–	3 each

Verified passing (what has been run and seen to pass, not everything that is registered):

- the 3 suites on `qemu-cortex-m0/m3/m4f/m7f`, `pico2-1cpu` and `2xcortex-m33`, with `cmsis-os-validator` at 60/60;
- `cortexm-pico2` 5/5 and `cortexm-pico2-rp2350b-psram` 2/2 emulated;
- `native`: all 14 host cases, in `gcc` and `sys`, debug and release;

- `cmsis-os-validator` 60/60 on the four Raspberry Pi platforms, emulated and on the boards, debug and release;
- on the Raspberry Pi Zero 2 W platforms, `mutex-stress`, `rtos-apis` and `smp_test0`, plus `smp_test1` and `smp_test3` on AArch32.

The other cases are registered but were not all run one by one. Every hardware case builds, and runs only when its board is connected.

11. Rules to keep

- **The code is the truth.** If a document disagrees with the files, the files win. The older documents in `docs/tests/` describe an earlier layout.
- **Commit new files with** `git add -A`. `git commit -a` leaves new folders out, and a fresh clone would then miss them.
- **Never commit** `build/`, `tests/xpacks/` or `node_modules/`. They are regenerated by `install`, `prepare` and `build`.
- A new board or test **must not change** results on boards that are already tested. Run their emulated sets again.
- Always pass the **full** case name to `-R`. It matches substrings, so a short name can run more than you meant.
- The whole history of how this framework was built is in `git log` of this file.